

Práctica: Clasificación con Yolo

Estado de madurez de Plátanos

Objetivo: Implementar, entrenar y evaluar un modelo de clasificación de imágenes utilizando la arquitectura YOLO (Ultralytics) para determinar el estado de madurez de un plátano.

1. Introducción

La clasificación de imágenes es una de las tareas fundamentales de la visión por computadora. A diferencia de la detección de objetos (que localiza elementos), la clasificación asigna una etiqueta a una imagen completa. En esta práctica, utilizaremos **YOLO11** para clasificar plátanos en diferentes estados: *maduras*, *verdes* o *pasadas*.

2. Recursos Necesarios

- **Dataset:** <https://universe.roboflow.com/zd7y6xooqnh7wc160ae7/banana-ripeness-classification>
 - **Librería:** ultralytics
 - **Documentación de referencia:** [Ultralytics Classify Docs](#)
 - **Clasificación con Yolo:** <https://docs.ultralytics.com/es/tasks/classify/>
-

3. Instrucciones de la Práctica

Paso 1: Configuración del entorno

Debes preparar tu cuaderno de trabajo instalando las dependencias necesarias.

Python

```
# Instalar la librería de Ultralytics
!pip install ultralytics
```

Paso 2: Descarga del Dataset

1. Entra al enlace de Roboflow proporcionado.
2. Haz clic en "Download Dataset".
3. Selecciona "Folder Structure"

Nota: Asegúrate de que el dataset se descargue con la estructura de carpetas: train, val, test.

Paso 3: Carga y Entrenamiento del Modelo

Utiliza un modelo preentrenado de clasificación (por ejemplo, yolo11n-cls.pt para velocidad o yolo11m-cls.pt para precisión).

Python

```
from ultralytics import YOLO

# Cargar un modelo de clasificación preentrenado
model = YOLO('yolo11n-cls.pt')

# Entrenar el modelo
results = model.train(
    data='RUTA_DE_TU_DATASET', # Ruta a la carpeta raíz del dataset
                                # descargado
    epochs=20,                  # Número de iteraciones
    imgsz=224,                  # Tamaño de la imagen
    device=0                    # Usar GPU (si está disponible)
)
```

Paso 4: Validación y Métricas

Una vez finalizado el entrenamiento, el modelo generará gráficas de precisión y pérdida en la carpeta runs/classify/train/.

- Analiza la **Matriz de Confusión**.
- Identifica qué clases presentan mayor dificultad de clasificación.

Paso 5: Inferencia con nuevas imágenes

Busca una imagen de una banana en Google Imágenes que el modelo no haya visto y realiza la predicción:

Python

```
# Realizar predicción
results = model('ruta/a/tu/imagen_nueva.jpg')
```

```
# Mostrar resultados
for result in results:
    print(result.probs) # Probabilidades de cada clase
```

4. Tareas a Entregar (Entregable)

Los alumnos deberán presentar un informe (o un Jupyter Notebook organizado) que contenga:

1. **Análisis del Dataset:** Indica cuántas imágenes hay por cada clase y cómo están distribuidas.
 2. **Configuración del Entrenamiento:** Justifica la elección del modelo (Nano, Medium, etc.) y el número de épocas.
 3. **Resultados Obtenidos:**
 - Captura de pantalla de la **Matriz de Confusión**.
 - Gráficas de **Loss** (pérdida) y **Accuracy** (precisión).
 4. **Prueba Real:** Muestra al menos 3 ejemplos de inferencia con imágenes descargadas de internet, indicando si el modelo acertó o falló.
 5. **Conclusiones:** Explica qué factores crees que afectan la precisión del modelo (iluminación, fondo de la imagen, etc.) y cómo se podría mejorar.
-

Formato de entrega: Archivo .ipynb y PDF con capturas del código y resultados.

5. Diferencias clave frente a Segmentación (Visto en clase):

No se usan archivos de etiquetas

- **En Detección de Objetos:** Necesitas un archivo `.txt` por cada imagen para guardar las coordenadas (`x`, `y`, `ancho`, `alto`) de cada caja. Como puede haber varios objetos en una imagen, el archivo es necesario.
- **En Clasificación:** La etiqueta es **toda la imagen**. No hay cajas que dibujar. Por lo tanto, la industria del Deep Learning (y YOLO lo sigue) utiliza un estándar llamado **Directory Labeling** (Etiquetado por Directorio).

¿Cómo sabe YOLO qué etiqueta tiene cada imagen?

YOLO deduce la etiqueta automáticamente basándose en el **nombre de la carpeta** donde está guardada la imagen.

La estructura que deben tener tus alumnos es esta (y es la que descarga Roboflow al elegir "Folder Structure"):

```
dataset/
├─ train/
│   ├─ ripe/    <-- YOLO asume que todas estas fotos son clase "ripe"
│   │   ├─ img1.jpg
│   │   └─ img2.jpg
│   └─ unripe/<-- YOLO asume que todas estas fotos son clase "unripe"
│       ├─ img3.jpg
│       └─ img4.jpg
├─ test/
│   ├─ ripe/
│   └─ unripe/
└─ valid/
    ├─ ripe/
    └─ unripe/
```

1. **Sin archivo .yaml:** En detección, usamos un `data.yaml`. En clasificación con Ultralytics, **no es obligatorio usar un .yaml**. Simplemente le pasas la ruta de la carpeta raíz del dataset:
 - o `model.train(data='ruta/del/dataset', ...)`
 - o YOLO buscará automáticamente las carpetas `train`, `val` y `test` dentro de esa ruta.
2. **Nombre de las clases:** Los nombres de las clases serán directamente los nombres de las carpetas.

Resumen

En clasificación no necesitamos archivos de texto para las etiquetas. El modelo es 'inteligente' y asume que si una imagen está dentro de la carpeta llamada 'madura', esa es su etiqueta. Por eso es vital que las carpetas estén bien nombradas y organizadas.